COSC 369 · SOFTWARE ENGINEERING I · MIDTERM UPDATE

Faculty Teaching & Office-Hours Schedule Generator

The schedule generator now comes with a question-answering layer, built with Python and Prolog.

Jaden Massey

Reading in the two files

File 1: the class schedule. One class per line.

```
CTEC 402.170|Software and OS Security (7R1)|Bengesi|MW|19:00|21:50|Online
```

code | title | professor | days | start | end | room. MW means Monday and Wednesday.

File 2: the professors. One professor per line.

```
Bengesi|Staphord Bengesi|CTEC|13:00|120|16:00
```

id | name | department | office hours end | evening block in minutes | evening starts at

Skips blank lines and notes

Splits each line at the | marks

Reports and skips a bad line

How office hours are worked out

1. Take one professor and one day.
2. Find that day's classes.
3. Sort them into daytime or evening, using the professor's evening start time.
4. After the last daytime class, office hours run until the professor's end time.
5. Before the first evening class, add a block of the professor's chosen length.
6. Keep every block between 8 AM and 6 PM. No class means no office hours.

Example: Prof. Harris on Tuesday

Ends at 1:00 PM. Evening starts at 4:00 PM. Evening block is 2 hours.

8:00 to 10:50 AM: CTEC 111.172 class

Office hours 10:50 AM to 1:00 PM

4:00 to 6:50 PM: CTEC 114.172 class

Office hours 2:00 to 4:00 PM

The Makefile builds everything

1. Compile

`src → obj`

Each piece of code becomes its own block. Only changed pieces are redone.

2. Bundle

`libschedgen.a`

The blocks are packed into one static library.

3. Link

`bin/schedgen`

The library joins the starting piece to make the program.

4. Run

`output/`

The program reads the two files and writes the schedule.

Command	What it does
make run	Builds if needed, then runs on the two files
make query	Asks the sample questions
make import	Turns the Excel schedule into the two files
make verify	Checks Prolog matches the program
make clean	Removes build files to start fresh

What changed

BEFORE

A report you read

The program built each professor's schedule and wrote it to a file. To find anything, you had to look through the report yourself.

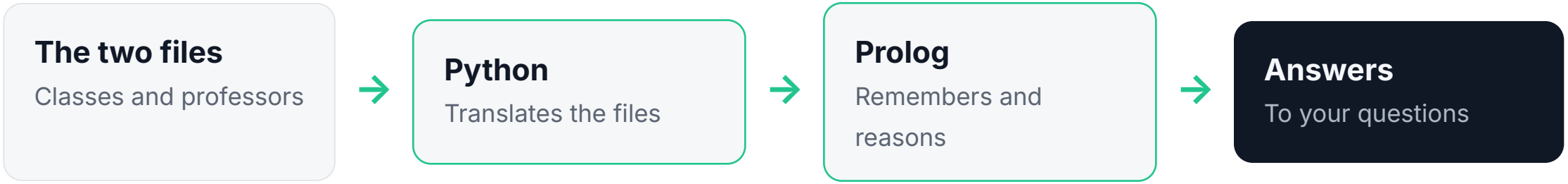
NOW

A schedule you can question

The same information is loaded into a question-answering system. Ask in a line, get an answer in a second.

The original program is unchanged. This is a new layer on top of it.

Two tools, two jobs



Python reads and translates. Prolog stores what is true and works out the answers.

Python turns the files into facts

It reads the same two files

No extra data to type in. Change a file and the facts update.

It writes one short fact per line

Each line states one simple thing that is true.

Nobody types facts by hand

That prevents typos and keeps the facts in sync with the source.

A class meets on Tuesday, 9:30 to 10:45 AM

```
meets('ctec 111.172', tuesday,
  480, 650, '111').
```

Prof. Harris's office hours preferences

```
policy(harris, 780, 120, 960).
```

Times are counted in minutes after midnight, so 480 is 8:00 AM.

Prolog stores facts and rules

Facts: what is true

Who teaches what, and when each class meets.

Rules: how to reason

How to work out new information from the facts.

One of the rules, in plain words

If a professor teaches a morning class that ends before their chosen cutoff, they have office hours from the end of that class until the cutoff.
If they teach an evening class, they have office hours right before it.

This is the same rule the original program uses, written a second way.

Ask a question, get an answer

Question	Answer
Who teaches CTEC 402.170?	Staphord Bengesi
What does Pierrot Chery teach?	CTEC 305.170, 305.180, 405.170
Which office hours are on Wednesday?	19 blocks, such as Carl Dixon 2 to 4 PM
Who has no class on Friday?	All 25 professors
Is Pierrot Chery in office hours Tuesday at 3 PM?	Yes

These are real answers from the CTEC Fall 2026 schedule. You can also type any question of your own.

Checking the work

62

office-hour blocks from the original program

62

office-hour blocks from Prolog

Match

same days, same times, every one

The same rule was written two different ways, in two different languages. When both give the same answer, it is strong evidence that neither one has a mistake.

Live demo

1. Build and run the original program
Produces the schedule report.

2. Ask the sample questions
Python and Prolog answer them.

3. Run the check
Confirms both versions agree.

4. Ask one of your own
Type any question live.

```
make run

make query

make verify

python3 scripts/query.py
  "teaches(P, 'ctec 402.170')"
```

What was added

- A Python script that turns the Excel schedule into the input files
- A second script that turns those files into facts
- Prolog rules that answer questions about the schedule
- A tool for asking questions, sample or your own
- A checker that confirms Prolog agrees with the original program
- New one-word commands in the Makefile
- An updated README with setup steps

Same data, same rules, and now you can simply ask.